Automated Medical Podcast Transcription and Topic Segmentation

1. Introduction

Podcasts and long-form audio content have become one of the most popular mediums for sharing information, discussions, and knowledge across various domains such as education, healthcare, business, and entertainment. However, extracting meaningful insights from lengthy audio recordings is often time-consuming, as users must listen to entire episodes to locate specific topics or information.

To address this challenge, this project presents an AI-driven Podcast Intelligence System that automatically converts audio recordings into text, segments the content into coherent topics, and enriches each segment with summaries, keywords, and emotional context. By leveraging recent advancements in speech recognition and natural language processing, the system enables efficient navigation, search, and understanding of podcast content.

The proposed system integrates OpenAI Whisper for speech-to-text transcription, transformer-based NLP models for topic segmentation and summarization, and an interactive Streamlit user interface for visualization and exploration. This approach significantly reduces manual effort, improves content accessibility, and enhances user experience by allowing users to quickly identify and focus on relevant sections of an audio recording.

2. Problem Statement

Podcasts and long-duration audio recordings contain valuable information, but extracting specific topics or insights from them is difficult and time-consuming. Users are often required to listen to the entire audio to locate relevant discussions, which reduces efficiency and accessibility. Traditional transcription tools provide raw text output but lack topic segmentation, semantic organization, emotional context, and interactive navigation.

Moreover, existing systems do not effectively support searching within audio content, identifying topic boundaries, or understanding the sentiment and emotion of speakers across different segments. This limitation makes it challenging for users to quickly analyze, summarize, or retrieve meaningful information from large volumes of audio data.

Therefore, there is a need for an intelligent, automated system that can transcribe audio, segment it into meaningful topics, analyze sentiment and emotions, and provide an interactive interface for efficient navigation and exploration of podcast content.

3. Objectives of the Project

The primary objective of this project is to develop an AI-powered system that intelligently processes podcast and long-form audio content to extract meaningful insights and enable efficient navigation.

The specific objectives of the project are:

- To automatically convert podcast audio into accurate text using speech-to-text technology.
- To perform audio preprocessing for noise reduction and quality enhancement.
- To segment long transcripts into coherent topic-based sections using semantic analysis.
- To generate summaries, titles, and key points for each identified topic.
- To extract important keywords to improve content understanding and searchability.
- To analyze sentiment and emotions present in the audio at both topic and segment levels.
- To enable semantic search and indexing for quick retrieval of relevant content.
- To design an interactive and user-friendly interface for topic navigation and visualization.
- To provide an AI-based question–answering assistant for conversational exploration of audio content.

4. Dataset Selection: Audio_recording_whisper

For the development and validation of this system, we selected the Audio_recording_whisper dataset from Kaggle as our primary corpus. This dataset consists of realistic doctor–patient conversational audio recordings, making it highly suitable for evaluating speech recognition and conversational intelligence systems.

- **Conversational Diversity:**

  The dataset captures natural doctor–patient interactions involving questions, explanations, pauses, and follow-ups. This diversity allows us to rigorously evaluate the Whisper model's ability to handle spontaneous speech, varying speaking styles, and conversational dynamics.

- **Domain-Specific Complexity:**

  Medical conversations include symptom descriptions, medical terminology, and contextual explanations. This semantic complexity effectively tests the performance of our summarization (DistilBART) and keyword extraction models in generating meaningful and concise insights.

- **Real-World Structure:**

  The structured yet conversational flow of clinical dialogues provides a reliable foundation for validating our topic segmentation logic, as discussions naturally transition between symptoms, history, diagnosis, and lifestyle factors.

5. Scope of the Project

The scope of this project is focused on the design and implementation of an AI-based audio intelligence system that processes long-form conversational audio and transforms it into structured, searchable, and insightful information.

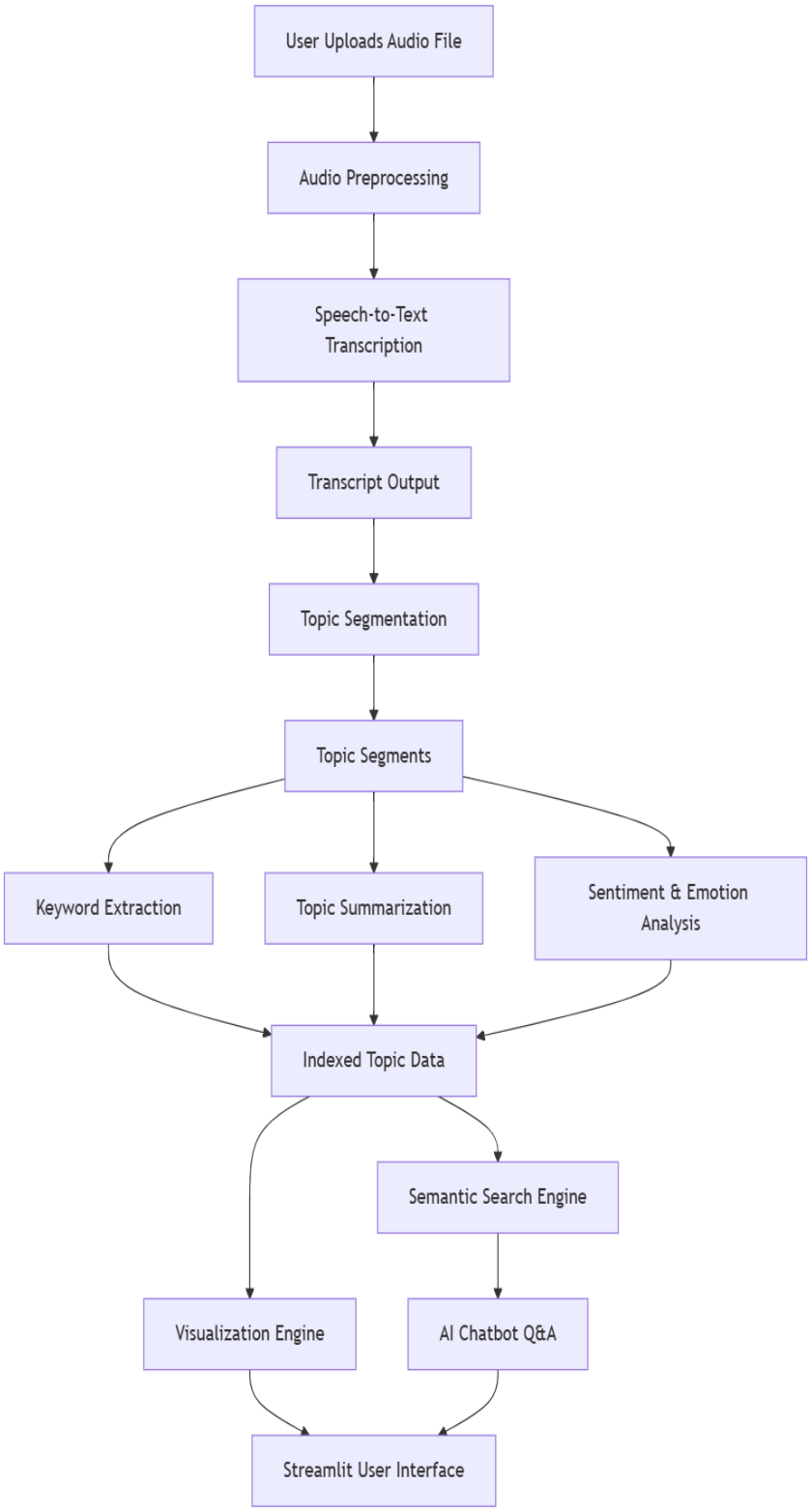
- **Audio Transcription:**
  The system supports automated transcription of conversational audio recordings using OpenAI Whisper, converting speech into accurate, timestamped text.

- **Topic Segmentation and Summarization:**
  The project includes semantic segmentation of transcripts into meaningful topics, along with the generation of summaries, titles, and key points for each segment.
- **Keyword and Emotion Analysis:**
  The system extracts important keywords and performs sentiment and emotion analysis to provide contextual understanding of each topic and speaker segment.
- **Semantic Search and Navigation:**
  Users can perform keyword-based and semantic searches to quickly locate relevant sections of the audio without listening to the entire recording.
- **Interactive Visualization:**
  The scope covers the development of an interactive Streamlit-based interface that visualizes topic timelines, emotion flows, and allows audio playback at the topic level.
- **Limitations:**
  The project is limited to offline audio processing, English-language content, and does not include real-time transcription or clinical diagnosis

6. Model Architecture

The system follows a **pipeline-based modular architecture**, where each stage performs a specific task and passes its output to the next stage.

High-Level Architecture Flow



This architecture improves scalability, maintainability, and clarity of system design.

7. Module Description

7.1 Audio Preprocessing Module

This module prepares raw audio files for accurate transcription. It utilizes LibROSA, SoundFile, and NumPy to perform audio resampling, mono conversion, noise reduction, and amplitude normalization. These preprocessing steps help reduce background noise and improve the overall quality of the audio signal, thereby enhancing transcription accuracy.

7.2 Transcription Module

The transcription module converts preprocessed audio into text using the OpenAI Whisper model (via Faster-Whisper), supporting multiple model sizes such as *tiny, base, and small*. The module generates timestamped speech segments, enabling precise alignment between the audio and textual content.

7.3 Topic Segmentation Module

This module identifies topic transitions within the transcript using semantic similarity–based segmentation. Sentence embeddings are generated using transformer models, and cosine similarity is applied across sliding windows to detect shifts in conversational context. Each resulting segment represents a meaningful and coherent discussion topic.

7.4 Intelligence Module (Keywords, Summaries, Sentiment)

- **Keywords:**
  Important keywords are extracted using KeyBERT and YAKE, leveraging semantic embeddings and statistical methods to identify contextually relevant terms for each topic.

- **Summarization:**
  The DistilBART Transformer model is used to generate concise summaries, titles, and bullet points for each topic segment, enabling quick understanding of long discussions.

- **Sentiment&Emotion:**
  Sentiment polarity and emotional states are analyzed using DistilBERT and DistilRoBERTa emotion models, providing both topic-level sentiment and segment-level

emotional context.

7.5 Frontend Visualization Module

The Streamlit-based frontend presents the processed results through an interactive dashboard that includes:

- Full, scrollable transcript with topic-wise organization
- Topic timeline visualization showing segment duration
- Emotion flow timeline illustrating sentiment and emotional intensity
- Semantic search with highlighted results and timestamp navigation
- Audio playback for individual topic segments

This module enables users to intuitively explore, analyze, and interact with long-form audio content.

8. Technologies Involved

Category	**Technologies**
Audio Processing	Librosa, PyDub, FFmpeg
Speech-to-Text	Whisper, Faster-Whisper
NLP	NLTK, SpaCy, Transformers
Topic Modeling	Sentence Transformers
Visualization	Plotly, Streamlit
Backend	Python
Storage	JSON, CSV

9. Project structure

```
project/
├── audio_raw/
│   └──                    # Original input audio files
├── audio_processed/
│   └──                    # Preprocessed and cleaned audio files
├── docs/
│   └──                    # Project documentation and reports
├── logs/
│   └──                    # Session-based log files
├── segments/
│   └──                    # Topic-segmented and enriched outputs
├── transcripts/
│   └──                    # Transcription outputs (JSON / TXT)
├── src/
│   ├── preprocessing.py          # Audio cleaning and normalization
│   ├── transcription.py          # Whisper-based ASR & speaker diarization
│   ├── segmentation.py           # Topic segmentation logic
│   ├── summarization.py          # Topic summarization (DistilBART)
│   ├── keyword_extraction.py     # Keyword extraction
│   ├── sentiment_analysis.py     # Sentiment and emotion analysis
│   ├── indexing.py               # Semantic indexing and search
│   ├── chatbot.py                # AI-based Q&A chatbot
│   ├── logger.py                 # Logging and session management
│   └── ui_app.py                 # Streamlit user interface
├── test_processed/
│   └──                                # Processed test artifacts
├── tests/
│   ├── test_chatbot.py                # Chatbot module tests
│   ├── test_detailed_emotion.py       # Detailed emotion analysis tests
│   ├── test_diarization_logic.py      # Speaker diarization logic tests
│   ├── test_emotion_manual.py         # Manual emotion validation tests
│   ├── test_indexing.py               # Indexing and search tests
│   ├── test_keyword_extraction.py     # Keyword extraction tests
│   ├── test_logger.py                 # Logger tests
│   ├── test_preprocessing.py          # Audio preprocessing tests
│   ├── test_segmentation.py           # Topic segmentation tests
│   ├── test_sentiment.py              # Sentiment analysis tests
│   ├── test_summarization.py          # Summarization module tests
│   └── test_transcription.py          # Transcription module tests
│
├── LICENSE                            # License information
├── README.md                          # Project overview and usage instructions
└── requirements.txt                   # Python dependencies
```

10. Prerequisites

Before setting up and running the Automated Audio Analysis and Podcast Intelligence System, ensure that the following software, tools, and knowledge requirements are met.

10.1 Software Requirements

- Python: Version 3.8 or higher
- Operating System: Windows / Linux / macOS
- FFmpeg: Required for audio format handling and processing
    - Ensure ffmpeg is added to the system PATH
- Git: For cloning the project repository

10.2 Python Libraries

All required Python dependencies are listed in requirements.txt.

They include libraries for:

- Audio processing
- Speech-to-text transcription
- Natural Language Processing (NLP)
- Machine Learning and Deep Learning
- Visualization and UI

Install them using:

pip install -r requirements.txt

10.3 Hardware Requirements

- Minimum RAM: 8 GB (Recommended: 16 GB for large audio files)
- Processor: Multi-core CPU
- GPU (Optional): NVIDIA GPU recommended for faster transcription and summarization

10.4 External Access (Optional but Recommended)

- Hugging Face Account & Token
    - Required for speaker diarization using pyannote.audio
    - Token must have read access

- Some models require accepting usage terms on Hugging Face

10.4 Basic Knowledge Requirements

- Basic understanding of Python programming
- Familiarity with command-line operations
- Introductory knowledge of:
  - Speech Processing
  - Natural Language Processing (NLP)
  - Machine Learning concepts (recommended, not mandatory)

10.5 Supported Audio Formats

- .mp3
- .wav
- .m4a

11. Installation Steps

Follow the steps below to set up and run the project.

Step 1: Install Python

1. Download Python from: https://www.python.org/downloads/
2. Install Python 3.8 or above
3. Important: Check "Add Python to PATH" during installation
4. Verify installation:

   python –version

Step 2: Install FFmpeg

1. Download FFmpeg from: https://ffmpeg.org/download.html
2. Extract the ZIP file
3. Add the bin folder path to System Environment Variables → PATH
4. Verify installation:

   ffmpeg -version

Step 3: Download / Clone the Project

Open Command Prompt and run:

git clone https://github.com/springboardmentor13579x-proj/Automated-Podcast-Transcription-and-Topic-Segmentation.git

cd project

(Or download the ZIP file and extract it)

Step 4: Create a Virtual Environment (Recommended)

python -m venv venv

venv\Scripts\activate

You should see (venv) in the command prompt.

Step 5: Install Required Libraries

pip install -r requirements.txt

Step 6: (Optional) Hugging Face Token for Speaker Diarization

1. Create an account at https://huggingface.co
2. Generate a Read Access Token
3. Accept the model agreements for pyannote.audio
4. Paste the token in the app when prompted

(Skip this step if diarization is not required)

Step 7: Run the Application

streamlit run src/ui_app.py

Step 8: Open in Browser

- The application will open automatically
- If not, open your browser and go to:

http://localhost:8501

11.1 Installation Complete

You can now:

- Upload audio files
- View transcripts and topics
- Search keywords
- Analyze sentiment and emotions

12. Usage

This section explains how to use the Automated Audio Analysis and Podcast Intelligence System after installation.

Step 1: Start the Application

Open Command Prompt, activate the virtual environment (if used), and run:

streamlit run src/ui_app.py

The application will open automatically in your default web browser.

Step 2: Upload an Audio File

1. Go to the Upload page in the application
2. Upload an audio file in one of the supported formats:
    - .mp3
    - .wav
    - .m4a
3. Alternatively, select an existing audio file from the server directory

Step 3: Process the Audio

1. Click Start Processing
2. The system will automatically execute the pipeline:
    - Audio preprocessing
    - Speech-to-text transcription
    - Topic segmentation
    - Keyword extraction

    - Summarization
    - Sentiment & emotion analysis
3. Progress will be displayed on the screen

Step 4: View Results

After processing is complete, you can:

- View the full transcript
- Navigate using the topic timeline
- Search for keywords or concepts
- Play audio for individual topic segments
- Analyze emotion and sentiment flow

Step 5: Use Semantic Search

1. Enter a keyword or phrase in the search box
2. Matching topic segments will be highlighted
3. Click a result to jump directly to that section

Step 6: Chat with the Podcast (Optional)

1. Open the AI Chatbot
2. Ask questions such as:
    - *"What is the main topic?"*
    - *"Summarize the podcast"*
    - *"What did they say about allergies?"*
3. The chatbot responds using relevant transcript segments

Step 7: Export Results (If Enabled)

- Download transcripts or analytics as CSV / JSON
- Export emotion flow data for further analysis

13.Troubleshooting

This section lists common issues users may face and their possible solutions.

13.1 Python Command Not Recognized

Issue:

python is not recognized as an internal or external command.

Solution:

- Reinstall Python
- Ensure **"**Add Python to PATH**"** is checked during installation
- Restart Command Prompt

13.2 FFmpeg Not Found

Issue:

Audio processing fails or FFmpeg-related error appears.

Solution:

- Verify FFmpeg installation
- Add the FFmpeg bin directory to System Environment Variables → PATH
- Restart the system

13.3 Application Runs Slowly

Issue:

Processing takes a long time for large audio files.

Solution:

- Use shorter audio clips
- Ensure sufficient RAM (8 GB minimum)
- Enable GPU support if available

13.4 Speaker Diarization Not Working

Issue:

Speakers are not separated in transcripts.

Solution:

- Provide a valid Hugging Face token
- Accept the required model agreements on Hugging Face
- Restart the application

13.5 Browser Does Not Open Automatically

Solution:

Manually open:

http://localhost:8501

14. Use Cases

The system can be applied across multiple domains:

14.1 Education

- Lecture transcription
- Topic-wise revision of recorded classes

14.2 Podcasts & Media

- Podcast summarization
- Topic navigation for long episodes

14.3 Healthcare

- Medical interview transcription
- Patient–doctor conversation analysis *(non-diagnostic)*

14.4 Corporate & Business

- Meeting analysis
- Interview review and insights

14.5 Research

- Speech and NLP research
- Behavioral and sentiment studies

15. Limitations and Challenges

Despite its effectiveness, the system has certain limitations:

- Accuracy depends on audio quality
- Background noise and overlapping speech reduce performance
- Limited support for non-English languages
- Long audio files increase processing time
- Emotion detection may not capture subtle expressions accurately

16. Future Enhancements

The system can be extended with the following features:

- Multilingual transcription support
- Real-time audio processing
- Improved speaker diarization accuracy
- Cloud-based deployment
- PDF and DOC report generation
- Integration with mobile applications
- Advanced analytics dashboards

17. References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever,
"Whisper: Robust Speech Recognition via Large-Scale Weak Supervision,"
OpenAI, 2022.

[2] N. Reimers and I. Gurevych,
"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,"
in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[3] M. Lewis, Y. Liu, N. Goyal, et al.,
"BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation,"
in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[4] B. McFee et al.,
"LibROSA: Audio and Music Signal Analysis in Python,"
in *Proceedings of the 14th Python in Science Conference*, 2015.

[5] Streamlit Inc.,
"Streamlit: A Fast Way to Build Data Apps in Python,"
[Online]. Available: https://docs.streamlit.io